

FI Clash 与 OPPO GDesk 企业 VPN 共存方案

目标：同时开启 FI Clash 和 GDesk，让 TeamTalk 正常工作，科学上网不受影响，不再需要每次使用 TeamTalk 前手动关闭 FI Clash。

本文分为两部分：

- **第一部分 原理与方案**：解释冲突原因和每处配置的作用，供你自己理解和排查。
- **第二部分 给 AI 助手的执行指令**：可整段复制发给 Codex 等具备电脑操作能力的助手，让它直接在你的机器上执行。

第一部分：原理与方案

结论

可以做到。FI Clash 的内核是 mihomo (Clash Meta)，原生支持把指定网段和网卡从代理中排除。但要稳定共存，必须**同时**处理三层，只改一层通常仍会失败：

层次	冲突原因	对应修改
路由层	TUN 虚拟网卡抢占了 GDesk 的路由	修改 B
DNS 层	内网域名被 fake-ip 劫持，返回假 IP	修改 C
规则层	内网流量被丢给了机场节点	修改 D
系统代理层	TeamTalk 读取系统代理设置后走了 Clash	修改 A

第一步：先花 3 分钟确认是哪一层在冲突

保持 GDesk 开启，然后分别测试：

1. FI Clash **只开系统代理、关掉 TUN** → TeamTalk 能用吗？
 - 能用：问题只在系统代理，做「修改 A」即可，最简单。
 - 不能用：进入第 2 步。
2. FI Clash **只开 TUN、关掉系统代理** → 不能用的话，就是路由/DNS 冲突，需要「修改 B + C」。

第二步：采集本机真实网络信息

下面配置中的网段、网卡名、DNS 都必须用实测值，不能猜。在 GDesk 未连接和已连接两种状态下各跑一遍，对比差异。

Windows：

```
route print -4          # 对比 GDesk 连接前后新增的路由条目 = 公司内网网段
ipconfig /all           # 找 GDesk 虚拟网卡名称和它下发的内网 DNS
Get-NetAdapter | Format-Table Name,InterfaceDescription,Status
nslookup <TeamTalk 服务器域名>
```

macOS：

```
netstat -rn -f inet     # 路由表
ifconfig                # 找 GDesk 的 utunX 网卡
scutil --dns             # 查看各网卡的 DNS
```

需要确定这五项：

- **A. 公司内网网段** —— GDesk 连接后新增的路由条目，可能是 `10.0.0.0/8`、`172.16.0.0/12`，也可能是若干条更精确的 `/16`、`/24`。
- **B. GDesk 虚拟网卡名称** —— Windows 是适配器名，macOS 是 `utunX`。
- **C. GDesk 下发的内网 DNS 服务器 IP**。
- **D. 物理上网网卡名称** —— Wi-Fi 或以太网，macOS 一般是 `en0`。
- **E. GDesk 是否是全隧道模式** —— 即是否添加了 `0.0.0.0/0` 默认路由。

另外打开 FiClash 的「连接」页面，启动 TeamTalk，观察它发起的连接命中了哪条规则、走了哪个节点。这是判断问题的最直接证据。

修改 A：系统代理绕过列表

位置：FiClash → 设置 → 覆写 → 网络 → 绕过域名（Bypass Domain）

加入公司域名后缀和内网网段（用实测值替换）：

```
*.oppo.com
*.myoas.com
10.*
172.16.*
192.168.*
```

注意：这个列表**只对系统代理模式生效**，TUN 模式下完全无效。很多人卡在这里，改了没用是因为同时开着 TUN。

修改 B：TUN 层不要抢 GDesk 的路由（最关键）

写入 FIClash 的「覆写（Override）」或配置文件，占位符替换成实测值：

```
interface-name: "<我的物理网卡名>"      # 如 "WLAN"、"以太网", macOS 一般 en0
tun:
  enable: true
  stack: mixed                          # 或 system
  auto-route: true
  auto-detect-interface: false          # 关键
  strict-route: false                  # 关键
  dns-hijack:
    - any:53
  exclude-interface:
    - "<GDesk 网卡名>"
  route-exclude-address:
    - <公司内网网段, 逐条列出>
    - <GDesk 网关服务器公网 IP>/32
```

三个要点：

1. **strict-route: false** —— 开启时 mihomo 会用防火墙规则强行把所有流量捞进 TUN，GDesk 再怎么加路由都没用，必定冲突。
2. **auto-detect-interface: false + 手工指定 interface-name** —— 否则 GDesk 一连上，内核检测到默认路由变成了 GDesk 虚拟网卡，会把机场流量也塞进公司隧道，结果是「全都不通」。
3. **必须排除 GDesk 网关服务器的公网 IP** —— 否则 GDesk 自身的握手包会被代理走，VPN 直接连不上。这个 IP 可在 GDesk 未连接时对其登录域名做 **nslookup** 得到。

FI Clash 特有的坑：桌面端默认用 UI 开关控制 TUN 参数，写在 yaml 里的 `tun` 段可能不生效。需要在设置里把 TUN 参数来源切换成「使用配置（config）」。改完后在 FI Clash 日志里找 `Tun adapter listening at ...` 那一行，确认实际生效的参数和你写的一致。

修改 C：DNS 分流

内网域名必须交给公司 DNS 解析，否则 fake-ip 会返回 `198.18.x.x` 的假 IP，TeamTalk 连过去必然超时。

```
dns:
  enable: true
  enhanced-mode: fake-ip
  fake-ip-filter:
    - "+.<公司域名后缀>"
  nameserver-policy:
    "+.<公司域名后缀>": "<GDesk 下发的内网 DNS IP>"
```

修改 D：规则层兜底

加在覆写规则列表的**最顶端**。规则自上而下匹配，放在后面会被 `GEOIP,CN,DIRECT` 之类抢先命中。

```
rules:
  - PROCESS-NAME,<TeamTalk 实际进程名>,DIRECT
  - DOMAIN-SUFFIX,<公司域名后缀>,DIRECT
  - IP-CIDR,<公司内网网段>,DIRECT,no-resolve
```

`PROCESS-NAME` 需要配置里有 `find-process-mode: strict` 才可靠。如果 TeamTalk 是 Electron 打包的，真正发请求的可能是子进程，此时应以域名和网段规则为主。

验证清单

同时开启 GDesk 和 FI Clash（TUN + 系统代理都开），逐项确认：

- ☐ TeamTalk 能正常登录和收发消息
- ☐ 科学上网正常（能访问 google.com）
- ☐ 国内网站正常（能访问 baidu.com）
- ☐ FI Clash 连接页显示 TeamTalk 的连接走 DIRECT，而非机场节点
- ☐ 重启 FI Clash、重连 GDesk 后配置依然生效

- ☐ TeamTalk 的语音/文件传输正常（走 UDP，TUN + fake-ip 下容易出问题）

兜底方案

如果三层都改完 TeamTalk 仍然不通，可能是 GDesk 属于驱动层劫持型企业 VPN（类似深信服 EasyConnect），检测到虚拟网卡存在就拒绝工作。这种情况无法纯靠配置解决，改用一键切换脚本，至少省掉手动点击：

```
# toggle-teamtalk.ps1 — 双击即可切换 FlClash 的 TUN + 系统代理
$p = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Internet Settings"
$on = (Get-ItemProperty $p).ProxyEnable
Set-ItemProperty $p ProxyEnable ([int](!$on))
if ($on) { Stop-Process -Name FlClash -ErrorAction SilentlyContinue }
else { Start-Process "C:\Program Files\FlClash\FlClash.exe" }
```

推进顺序建议

先做诊断确认冲突层次，再按 B → C → D 依次添加，每加一层就测一次 TeamTalk。这样出问题时能立刻知道是哪一层写错了。

第二部分：给 AI 助手的执行指令

以下内容可整段复制，发给 Codex 等具备电脑操作能力的 AI 助手。

任务：让 FlClash 与公司 VPN（OPPO GDesk）共存，无需每次关闭 FlClash

背景

我的机器上有两个网络工具：

1. FlClash（mihomo/Clash Meta 内核的代理客户端），日常科学上网用。
2. GDesk（OPPO 的企业 VPN 客户端），公司内部 IM 软件 TeamTalk 依赖它。

现状问题：只有关掉 FlClash、单独开 GDesk 时，TeamTalk 才能正常使用。

目标：改造 FIClash 的配置，让两者可以同时开启，TeamTalk 正常工作，同时我的 科学上网也不受影响。改完之后我不需要再手动开关 FIClash。

硬性约束（请严格遵守）

- 动任何配置文件之前，先完整备份一份，并把备份路径告诉我。
- 不要删除或修改我的机场订阅本体。所有自定义内容都通过 FIClash 的「覆写（Override）」功能注入，或写在独立的本地配置里。
- 下面 YAML 里所有尖括号占位符（如 `<GDesk网卡名>`）都必须用阶段 1 实测到的 真实值替换，禁止照抄占位符，也禁止凭空猜测网段。
- 每个阶段结束后停下来，把采集到的关键信息和你的判断结论告诉我，再继续。
- 如果某一步改完之后网络反而全断了，立刻回滚到备份并告诉我。

阶段 0：环境探测

确认我的操作系统（Windows / macOS）、FIClash 的版本和配置文件所在目录。

- Windows 常见路径：`%APPDATA%\FIClash` 或安装目录下的 data 文件夹
- macOS 常见路径：`~/Library/Application Support/FIClash`

把找到的配置文件完整列出来给我看。

阶段 1：采集真实网络信息

先让 GDesk 处于「未连接」状态，跑一遍下面的命令并保存输出；然后连上 GDesk，再跑一遍，对比两次差异。

Windows：

```
route print -4
ipconfig /all
Get-NetAdapter | Format-Table Name,InterfaceDescription,Status
```

macOS：

```
netstat -rn -f inet
ifconfig
scutil --dns
```

需要从对比结果中确定这五项，逐条列给我：

- **A.** GDesk 连接后新增的路由条目，这就是公司内网网段（可能是 `10.0.0.0/8`、`172.16.0.0/12` 之类，也可能是若干条更精确的 `/16`、`/24`）。
- **B.** GDesk 虚拟网卡的准确名称（Windows 是适配器名，macOS 是 `utunX`）。
- **C.** GDesk 下发的内网 DNS 服务器 IP。
- **D.** 我的物理上网网卡名称（Wi-Fi 或以太网，macOS 一般是 `en0`）。
- **E.** 特别注意 GDesk 是否添加了 `0.0.0.0/0` 默认路由（全隧道模式），如果是请明确告诉我，这会影
响后续策略。

另外，请找出 TeamTalk 的实际可执行文件名和它连接的服务器域名/IP：

- Windows： `Get-Process | Where-Object {$_.Name -like "*eam*alk*"}`
- 启动 TeamTalk 后用 `netstat -ano`（Windows）或 `lsof -i`（macOS）看对端地址
- 也可以打开 FIClash 的「连接」页面，启动 TeamTalk，看它发起了哪些连接、命中了哪条规则、走了哪个节点。这是最直接的证据，请截图或抄录给我。

阶段 2：分层诊断

保持 GDesk 连接，按顺序测：

- **测试 1：** FIClash 只开「系统代理」，关闭 TUN → TeamTalk 能用吗？
- **测试 2：** FIClash 只开「TUN」，关闭系统代理 → TeamTalk 能用吗？

判定：

- 测试 1 失败、测试 2 成功 → 系统代理层冲突，只需执行【修改 A】。
- 测试 1 成功、测试 2 失败 → 路由/DNS 层冲突，执行【修改 B】【修改 C】。
- 两个都失败 → 三层都做，执行【修改 A】【B】【C】【D】。

把测试结论告诉我再往下走。

【修改 A】系统代理绕过列表

在 FIClash 的设置 → 覆写 → 网络 → 绕过域名（Bypass Domain）里，加入公司域名后缀和内网网段（用阶段 1 查到的实际值），例如：

```
*.oppo.com
*.myoas.com
10.*
172.16.*
192.168.*
```

注意：这个列表只对系统代理模式生效，TUN 模式下无效，不要指望它解决 TUN 的问题。

【修改 B】TUN 层：不要抢 GDesk 的路由（最关键）

把下面配置写入 FIClash 的覆写或配置文件，占位符全部替换成实测值：

```
interface-name: "<我的物理网卡名>"
tun:
  enable: true
  stack: mixed
  auto-route: true
  auto-detect-interface: false
  strict-route: false
  dns-hijack:
    - any:53
  exclude-interface:
    - "<GDesk网卡名>"
  route-exclude-address:
    - <阶段1查到的公司内网网段，逐条列出>
    - <GDesk网关服务器的公网IP>/32
```

要点说明（请理解后再改，不要机械照搬）：

- `strict-route` 必须为 `false`。开启时 `mihomo` 会用防火墙规则强行捞走所有流量，GDesk 加多少路由都没用。
- `auto-detect-interface` 必须为 `false` 并手工指定 `interface-name`。否则 GDesk 一连上，内核检测到默认路由变成 GDesk 虚拟网卡，会把我的机场流量也塞进公司隧道，结果是「全都不通」。
- GDesk 网关服务器的公网 IP 必须排除，否则 GDesk 自身的握手包会被代理走，VPN 直接连不上。这个 IP 可以在 GDesk 未连接时对它的登录域名做 `nslookup` 得到。

FIClash 特有的坑，请务必检查：桌面端默认用 UI 开关控制 TUN 参数，写在 yaml 里的 `tun` 段可能不生效。需要在设置里把 TUN 参数来源切换成「使用配置（config）」，yaml 才会被采纳。改完后到 FIClash 日志里找 `Tun adapter listening at ...` 那一行，确认实际生效的参数和我写的一致，把这行日志贴给我。

【修改 C】DNS 分流

内网域名必须交给公司 DNS 解析，否则 `fake-ip` 会返回 `198.18.x.x` 的假 IP，TeamTalk 连过去必然超时：


```
dns:
  enable: true
  enhanced-mode: fake-ip
  fake-ip-filter:
    - "+.<公司域名后缀>"
  nameserver-policy:
    "+.<公司域名后缀>": "<GDesk下发的内网DNS IP>"
```

【修改 D】规则层兜底

在覆写规则列表的最顶端加入（顺序很重要，规则自上而下匹配，放后面会被 **GEOIP,CN,DIRECT** 之类抢先命中）：

```
rules:
  - PROCESS-NAME,<TeamTalk实际进程名>,DIRECT
  - DOMAIN-SUFFIX,<公司域名后缀>,DIRECT
  - IP-CIDR,<公司内网网段>,DIRECT,no-resolve
```

PROCESS-NAME 需要配置里有 **find-process-mode: strict** 才可靠。如果 TeamTalk 是 Electron 打包的，真正发请求的可能是子进程，此时以域名和网段规则为主。

阶段 3：验证

同时开启 GDesk 和 FIClash（TUN + 系统代理都开），逐项验证并把结果告诉我：

1. TeamTalk 能正常登录和收发消息。
2. 我的科学上网正常（能访问 google.com）。
3. 国内网站正常（能访问 baidu.com）。
4. 在 FIClash 连接页确认 TeamTalk 的连接走的是 DIRECT，而不是某个机场节点。
5. 重启 FIClash、重连 GDesk，确认配置持久生效，不是一次性的。
6. 特别测一下 TeamTalk 的语音/文件传输（走 UDP），TUN + fake-ip 下 UDP 容易出问题。

阶段 4：兜底方案

如果三层都改完 TeamTalk 仍然不通，说明 GDesk 可能是驱动层劫持型的企业 VPN（类似深信服 EasyConnect），检测到虚拟网卡存在就拒绝工作，这种情况无法纯靠配置解决。此时请帮我做一个一键切换脚本放到桌面，双击即可切换 FIClash 的 TUN + 系统代理状态，至少省掉手动点击。并明确告诉我是这个原因导致的。

输出要求

最后给我一份总结：改了哪些文件、每处改动的作用、备份在哪、如何一键回滚。

附录：关键参数速查

参数	推荐值	作用
tun.strict-route	false	关闭强制接管，是共存的前提
tun.auto-detect-interface	false	防止内核把 VPN 虚拟网卡当出口
interface-name	物理网卡名	固定代理流量的出口
tun.exclude-interface	GDesk 网卡名	该网卡的流量不进 TUN
tun.route-exclude-address	内网网段 + VPN 网关 IP	这些目标不走 TUN 路由
dns.fake-ip-filter	公司域名后缀	内网域名不用假 IP
dns.nameserver-policy	公司域名 → 内网 DNS	内网域名走公司 DNS 解析
find-process-mode	strict	让 PROCESS-NAME 规则可靠生效